

MrScraper

AI Engineer — Take Home Test

Price Intelligence & Anomaly Detection

Role	AI Engineer — Price Intelligence & Anomaly Detection
Deadline	One week from the date of receipt
Submission	Public GitHub repository; reply-all to the original email with the link
Dataset	Train set: 306,226 rows Test set: 76,255 rows across 16 days (3 days shared upfront; 13 days shared at interview)

Introduction

At MrScraper, we continuously scrape millions of product listings across e-commerce platforms, capturing price changes, discount patterns, promotional events, and shop-level signals over time. This historical data forms the backbone of our price intelligence capabilities.

This take-home test challenges you to work with a real-world snapshot of that data and build a robust price prediction pipeline. Your solution will be evaluated on its technical depth, analytical rigor, and how thoughtfully you reason about the problem.

Background & Problem Statement

There are moments when an e-commerce platform becomes temporarily inaccessible — whether due to anti-scraping countermeasures, platform downtime, or network-level blocks. When this happens, real-time price data is unavailable and we lose visibility into current pricing for the thousands of products we track.

To mitigate this, a small team can manually collect pricing data for a limited number of products — around 100 items — on the affected day. The challenge then becomes: given this tiny anchor

set of manually collected prices and a rich historical dataset of past scrapes, can we accurately reconstruct the prices for the thousands of other products we care about on that same day?

Your task is to build a model that addresses exactly this scenario:

- You have a large historical training dataset (306,226 rows) of past scraping results.
- On the prediction day, you are given 100 manually collected product prices as a calibration anchor set.
- Your model must leverage both the historical data and this anchor set to predict prices for the remaining products in the test set — all of which have appeared in the training data before.

The key insight is that the 100 anchor samples are not just evaluation data — they are a signal. A well-designed model should use them to calibrate its predictions for the day in question (e.g. adjusting for platform-wide price shifts, promotional events, or currency changes on that day).

Dataset Description

You are given a training dataset of 306,226 rows. The test file spans 16 days and contains 76,255 rows in total. Within each day, 100 rows have the price already filled in as manually collected anchor samples — 1,600 anchor rows across all 16 days. The remaining rows have the price left blank and must be predicted.

To help you verify your inference pipeline before the interview, 3 days of the test file are shared. You are free to use these to validate that your code runs correctly and produces sensible predictions. The remaining 13 days will be shared at the start of your interview session, at which point you will be expected to run your pipeline and submit the fully completed file. All products in the test file appear in the training data and share the identical schema described below.

Train set:

<https://drive.google.com/file/d/1ZOYuyrcBJF7fvnG6kBva1m8og5urXHjU/view>

Test set - 3 days:

https://drive.google.com/file/d/1Ni2aBrOaV1YEWspZVmBw_HV1a7M37cd/view

Column Reference

Column	Description
<code>capturedAt</code>	Timestamp when the data was scraped from the e-commerce platform.
<code>shopId</code>	Unique identifier of the seller/shop on the platform.

Column	Description
itemId	Unique identifier of the product/item listing.
modelId	Unique identifier of the specific product variant or model (e.g. size, color).
price	Current listed price of the item (in the platform's base currency unit).
priceBeforeDiscount	Original price before any discount is applied. 0 if no discount exists.
promotionId	Identifier of the active promotion applied to the item. 0 if none.
cat_id	Category identifier that classifies the product into a marketplace category.
stock	Current available stock quantity of the item/model.
normal_stock	Normal (non-flash sale) stock level for the item/model.
raw_discount	Absolute discount value in the platform's base currency unit.
show_discount	Discount percentage displayed to the buyer (0–100).
brand	Brand name associated with the product.
is_free_shipping	Boolean flag indicating whether free shipping is offered (t/f).
is_pre_order	Boolean flag indicating whether the item is a pre-order product (t/f).
item_price_min	Minimum price among all variants/models of the same item listing.
item_price_max	Maximum price among all variants/models of the same item listing.
review_rating	Average rating score given by buyers, typically on a scale of 0–5.
total_rating_count	Total number of ratings received by the item.
cmt_count	Total number of written comments/reviews for the item.
shop_rating	Overall rating score of the shop, typically on a scale of 0–5.
shop_response_rate	Percentage rate at which the shop responds to buyer inquiries (0–100).
shop_follower_count	Total number of users following the shop.
is_official_shop	Boolean flag indicating whether the shop is an officially verified brand store (t/f).
is_verified	Boolean flag indicating whether the shop has been verified by the platform (t/f).
is_preferred_plus_seller	Boolean flag indicating whether the shop holds Preferred Plus seller status (t/f).

Note: Prices and currency values are stored in the platform's smallest currency unit (e.g., IDR in units of 1, not thousands). You may choose to normalise these as part of your preprocessing.

Task Description

Build a price prediction system that can reconstruct product prices on a day when scraping was unavailable, using historical data and a small anchor set of manually collected prices. The target variable is price. You are expected to explore and compare two modelling approaches:

Approach 1 — Global Marketplace Model

Train a single model on the entire historical dataset. This model should generalise across all shops, items, and categories, acting as a marketplace-wide price predictor. Use the 100 anchor samples to calibrate or adjust its outputs for the prediction day.

- Suitable as a baseline and for products with sparse history
- The anchor set can be used to detect and correct for any systematic day-level price shift
- Evaluate its performance on your local validation split before seeing the test data

Approach 2 — Shop / Product Level Model

Train models (or a conditioned model) that group by shopId, itemId, and/or modelId as a primary key. Each shop may have its own pricing strategy, discount rhythm, or seasonal behaviour. The 100 anchor samples should inform per-entity calibration where possible.

- More personalised — captures shop-specific and product-specific pricing logic
- Expected to outperform the global model for shops and products with sufficient history
- Consider how you handle entities with very few historical observations

You are encouraged to explore both approaches, report your validation accuracy findings, and reflect on when each would be more suitable in a real production outage scenario.

Evaluation Criteria

Your submission will be assessed across the following dimensions:

1. Prediction Accuracy

- Use appropriate regression metrics such as MAE, RMSE, or MAPE
- Report metrics separately for the global model and the per-shop/product model
- Clearly document your validation methodology (e.g. time-based split simulating an outage day)

2. Anchor Set Utilisation

- How effectively does your model use the 100 anchor samples to calibrate predictions?
- Does incorporating the anchor set meaningfully improve accuracy over a baseline that ignores it?
- Discuss your strategy: global bias correction, per-category adjustment, fine-tuning, or other approaches

3. Feature Engineering

- How well you extract signals from temporal features (capturedAt)
- How you handle categorical identifiers (shopId, itemId, modelId, cat_id, brand)
- Creative use of derived features (e.g. discount depth, shop engagement score, price momentum)

4. Modelling Approach

- Justification of model choices (e.g. tree-based, deep learning, time series)
- How you handle the multi-level nature of the data (global vs. per-entity)
- Treatment of outliers and missing values

5. Analysis & Insights

- Comparison between Approach 1 and Approach 2 — where does each perform better?
- What patterns or anomalies did you observe in the data?
- Any unexpected findings worth highlighting

6. Code Quality & Reproducibility

- Clean, well-structured Python code
- Clear README with setup instructions and how to reproduce results
- Reproducible random seeds and environment specification (requirements.txt or similar)

Submission Guidelines

Please submit your solution by pushing your work to a public GitHub repository and replying to this email with the link within one week of receiving this test.

Your repository should include:

1. A README.md explaining your approach, how to run the code, and a summary of your findings
2. All code used for data preprocessing, feature engineering, and model training
3. A results summary (notebook or markdown) comparing the two approaches with validation metrics
4. Your trained model artifacts or clear instructions on how to reproduce them
5. A requirements file (requirements.txt, environment.yml, or similar)

You will receive 3 days of the test file alongside this document to verify your inference pipeline works as expected. At the start of the interview session, the remaining 13 days will be shared. You will be expected to run your pipeline on the full test file, submit the completed CSV with all missing prices filled in, and walk through your approach and findings live.

Grading Overview

Submissions will be scored across two tiers:

Tier	Approach	Description
Tier 1	Global Marketplace Model	A single model trained on all historical data, calibrated using the 100 anchor samples to predict prices across the entire marketplace on the outage day.
Tier 2	Shop / Product Level Model	Models conditioned on shopId, itemId, and/or modelId to capture entity-specific pricing patterns, with anchor-set calibration applied at the shop or product level. Expected to yield higher accuracy where sufficient history exists.

Both tiers are recommended to be attempted. You are encouraged to reflect on the accuracy difference between the two in your submission, and discuss when each approach would be more suitable in a production setting. You are welcome to propose even another approach!

Tips & Guidance

- Simulate the outage scenario during validation: Hold out one day's data entirely as your validation set and treat a random 100 samples from it as your anchor set. This mirrors the real test conditions.
- Anchor set calibration: Think about what the 100 anchor samples tell you. Do prices shift uniformly across the platform? By category? By shop tier? Use this insight to calibrate your predictions.
- Time-aware features: Since capturedAt is available in the training data, extract temporal signals carefully. Avoid leakage — your model should not assume knowledge of the prediction day's date beyond what the anchor set reveals.
- Cold start: Consider how your Tier 2 model handles items or shops with very few historical observations — this is a realistic production concern.
- Outliers: Price data often contains outliers due to data entry errors or flash sales. Document how you handle them.
- Explainability: If time permits, include feature importance analysis or SHAP values to explain your model's predictions.

Final Notes

We are not only evaluating your model performance — we are also assessing how you think through a real-world ML problem. Clear communication, thoughtful trade-off discussions, and honest reporting of limitations are just as important as your metrics.

If you have any questions or need clarification on the dataset or requirements, feel free to reach out before your submission deadline.

We look forward to reviewing your work and discussing it with you during the interview. Best of luck!

MrScraper — Confidential Hiring Document